Two intellectual traditions both study sequential decision-making under uncertainty, but they descend from different intellectual traditions. The first is fundamentally an *inference culture*. Its central task is to understand the world. A "model" in this tradition is a specification of preferences, beliefs, constraints, and an equilibrium concept. The RL tradition is instead a *control culture*. Its central task is to act in the world. An RL researcher's "model" is a transition kernel $P(s'|s, a)$ and a reward function $r(s, a)$. These are different mathematical objects serving fundamentally different scientific purposes.

The inference tradition concentrates its effort on specifying and estimating the objective function and the law of motion that governs the environment. Is the welfare criterion correct? Does the model capture how agents actually behave? What happens if the dynamics are misspecified? The entire structural econometrics enterprise (demand estimation, dynamic discrete choice) is about recovering these primitives from data. The optimal policy is a byproduct that falls out once the primitives have been identified; an economist would not publish a paper claiming "here is a good controller" without first arguing that the model is the right description of reality. The control tradition inverts this emphasis. Engineers take the objective and the dynamics as given (the physics is the physics, the spec sheet says "minimize tracking error") and focus entirely on whether the optimal policy can be computed, approximated, and deployed under real-time and robustness constraints. The entire controls enterprise (PID, LQR, MPC, RL) is about computing and implementing the policy under different assumptions about what the agent knows and what it can compute. An engineer would not spend a year debating whether the cost function is correct; the plant manager specified the setpoint and that closes the question.

The two cultures maintain different relationships with data. The inference culture works primarily with observational data, and endogeneity is the central obstacle. Identification is its defining challenge which involves dealing with functional forms, equilibrium definitions, instrumental variables, regression discontinuities, and natural experiments. RL researchers have traditionally enjoyed what might be called simulator omnipotence. They often own the data-generating process and can create the variation in their data and fit any manner of nonparametric functions through millions of simulated samples. They face computational rather than statistical constraints. This difference shapes the cultures, from algorithm design to what counts as a valid result and what is the standard

of evidence.

The two fields also differ in their treatment of human agency and in the meaning of basic terms like "agent," "learning," "model," and "policy," as detailed in the subsections that follow.

The two cultures have a closely related and shared vocabulary but with divergent meanings. And this creates persistent confusion for researchers crossing from one field to the other. This chapter attempts to provide a systematic translation and notes common sources of confusion.

0.1 Core Distinctions

RL organizes its core problems along axes that do not map directly onto well known categories. In RL, *Prediction* refers to estimating $V^\pi(s)$ or $Q^\pi(s, a)$ for a fixed policy π (policy evaluation), not forecasting observable variables. *Control* refers to finding the policy π^* that maximizes expected discounted return (policy optimization), not the inclusion of regressors. Because prediction concerns evaluating a specific policy, a closely related question is whether the data was generated by the policy being studied. *On-policy* methods evaluate and improve the same policy that generates the data. *Off-policy* methods learn about a target policy π from data generated by a different behavioral policy μ. This distinction is central to causal inference (Section **??**), where off-policy evaluation is precisely counterfactual policy evaluation.[1]

A separate question concerns how the algorithm accesses experience. *Online* RL learns while interacting with the environment, collecting new data as a consequence of its own actions. *Offline* RL (also called batch RL) learns exclusively from a fixed dataset of previously collected transitions, with no ability to gather additional samples. The offline setting is closer to standard empirical work, where the dataset is given and the analyst cannot run new experiments; the online setting corresponds more closely to adaptive experimental design or sequential decision problems. Note that "online" in RL carries no timing constraint; it means only that the agent generates fresh experience. *Real-time* RL, by contrast, imposes hard deadlines on the perception-action loop, as in robotics

[1] "Counterfactual" here is used in the interventional sense, asking "what would happen if we deployed policy π instead of μ?" This is distinct from the structural counterfactual in **?**, which conditions on the specific realized trajectory and asks what would have happened to *this* individual under a different action, requiring a fully specified structural causal model rather than just the observational distribution under μ.

or mechatronics. Every real-time RL system is online, but most online RL (games, recommender systems) are not real-time.

Aside from data access is the question of what the algorithm does with that data. In RL, the word *model* refers strictly to the environment's dynamics, the transition kernel $P(s'|s,a)$ and reward function $r(s,a)$. A *model-based* RL algorithm explicitly constructs or is given a mathematical representation of P and r, then computes a policy by planning through that representation (for example, using a simulator or the known rules of a game, as in AlphaZero). A *model-free* algorithm computes the value function or policy directly from experienced transitions without ever building an explicit representation of the transition probabilities. Crucially, "model-free" need not mean the algorithm lacks complete access to any model of the environment. A model-free algorithm could interact with a simulator that internally implements a complete computerized model of the environment; the distinction is that the algorithm never extracts or plans through the transition probabilities, treating the simulator as a black box that merely returns sample transitions. However, it is possible that a "model-free" algorithm could also be implemented "in-field" where there is genuinely no access to a "model" of the environment but only direct access to the environment itself (for reasons discussed later, this is rarely done).

One might believe that an analog to "model-free" RL algorithms might be reduced-form or a statistical model that only requires a few assumptions (exogeneity, etc.) to make inference on parameters of interest. But this is a false analogy, as regardless of "model-free" or "model-based" we are talking about learning optimal policies within a MDP (this itself demands a certain structure on the problem including the Markovian assumption that the state contains all relevant information). One could believe that "model-based" algorithms are analogous to structural estimation, which involves the specification of preferences, technology, beliefs, and an equilibrium concept, all of which give rise to specific transition dynamics and reward functions. This is also a false analogy. In RL terminology, for an algorithm to be "model-based", the conditions are very precise, all we need is a given representation of P and r (regardless of whether you do or don't make any structural assumptions about human or firm behavior or their interactions). All this to say, in RL, the model generally only refers to transition matrices P and rewards r within an MDP; whereas in the inference tradition it refers to a set of agents, preferences,

exclusion restrictions, and equilibrium concepts. Therefore, it is possible to do structural estimation of a rich model with a lot of structure but by using a "model-free" RL algorithm as a computational tool to find the equilibrium, as in Section **??**.

Every RL system passes through two phases. In the *training phase*, the agent interacts with an environment (simulated or real) and updates its parameters. In the *execution phase* (also called the deployment phase), the policy is frozen and used to make decisions without further updates. This distinction is critical for interpreting "online." Online training in RL almost always takes place inside a simulator (and not in the "real world"). AlphaGo Zero trained online through millions of self-play games; when it faced Lee Sedol, its weights were frozen and it was purely executing its trained policy. Some deployed systems in Section **??** followed this pattern cleanly; DiDi's dispatch system trained a value function from historical trip data, then deployed with fixed weights. Others blur the boundary. The hotel revenue management system in Section **??** updated Q-values from realized returns after each completed episode during live operation, making it an *in-field* learner rather than a frozen executor.[2] Bandit algorithms (Section **??**) also learn in-field by design, updating demand estimates from real customer responses during deployment.

A related terminological hazard concerns the word *inference*. In machine learning, "inference" refers to executing a frozen model, a forward pass producing outputs from inputs. In statistics, "inference" means statistical inference, the construction of standard errors, confidence intervals, and hypothesis tests. This survey uses "inference" exclusively in the statistical sense and "execution" or "deployment" when referring to applying a trained model (whether in a computer or in field). Established terms such as "Bayesian inference" and "variational inference," which refer to inference about parameters or distributions, are used where appropriate. The key takeaway is that most RL convergence results and sample complexity guarantees refer to the training phase, and interpreting them as claims about deployed performance requires additional argument. Table 1 summarizes these distinctions.

A typical applied RL pipeline moves through the grid sequentially, from pre-training on historical logs (middle-left) to refinement in a simulator (top-left) to deployment with frozen weights (bottom-right). Bandits illustrate this fluidity; even a bandit algorithm

[2] "In-field" is not standard RL vocabulary. We introduce it here to distinguish live-market online learning, where exploration has real financial cost, from the far more common case of online learning inside a simulator.

Table 1: The reinforcement learning lifecycle grid. Most RL research operates in the top-left cell. Readers from the inference tradition typically picture the bottom-left when they hear "online."

	Training (parameters updated)	Execution (parameters frozen)
Simulator	· AlphaGo Zero self-play · RL solver in structural estimation · Bandit calibration via simulation	· Policy benchmarks on synthetic environments
Historical data	· RLHF reward model from preference logs · Causal OPE from observational records · Logged-bandit policy learning	· Off-policy evaluation of a target policy
Live market ("in-field")	· Bandit pricing experiments · Hotel RM Q-learning from live episodes	· DiDi dispatch with frozen weights · AlphaGo vs. Lee Sedol

that will ultimately learn in-field is typically calibrated in simulation and tuned on historical logs before any live deployment, because in-field exploration incurs real financial cost. The systems that do operate in-field arrive with exploration parameters, initial policies, and demand priors shaped by extensive offline preparation.

The Tmall e-commerce pricing project of Liu et al. (2019) (Section **??**) illustrates this migration concretely. The team pre-trained a DQN from logged specialist pricing decisions (historical data, training), then ran offline evaluation of the candidate policy on held-out transaction logs before any live deployment (historical data, execution). The evaluated policy was deployed for 15 to 30 day field experiments on live Tmall traffic, with the agent receiving reward and observation signals from the market environment (live market, execution and training). Liu et al. (2019) note that no accurate simulator exists for e-commerce pricing, so the project skipped the simulator row entirely, jumping from historical pre-training directly to live deployment. Not every application traverses all six cells of Table 1, but the grid clarifies which cells a given project could be working on.

0.2 Overlapping Terminology

The word *agent* refers to different entities. In the inference tradition, an agent is always a human decision-maker (a consumer, worker, or firm) or a social planner whose choices are

the object of study. In RL, "the agent" is the learning algorithm itself, or more precisely an algorithm deployed on behalf of a human decision-maker, and the human, if present at all, is part of the environment providing reward signals.

In RL the *environment* is a formal object encompassing everything outside the agent (the DGP, other agents, market clearing conditions), whereas in the inference tradition "environment" refers more loosely to market structure or institutional rules.[3]. RL speaks of *rewards* where the other tradition speaks of *utility*. The mapping is not exact: a reward $r(s, a)$ is a known, externally specified function that the algorithm maximizes, whereas utility $u(x, d)$ in the inference tradition is a primitive of preferences that the analyst must recover or estimate from observed choices.

In RL, the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ is the discounted sum of future rewards from time t onward, the random variable whose expectation defines the value function. The RL usage is closer to what is called the "present discounted value" of a stream of payoffs. A single complete sequence of interactions from an initial state to termination, $(s_0, a_0, r_1, s_1, \ldots, s_T)$, is called an *episode* (synonymously, *trajectory* or *rollout*).[4] Where a statistician speaks of the *outcome*, meaning the dependent variable Y in a regression, RL has no single analog: the reward r_t is the per-period outcome, the return G_t is the cumulative outcome, and the value function $V^\pi(s)$ is the expected cumulative outcome conditional on state.

Learning carries many distinct meanings in the two fields. In decision theory (Bayesian learning, adaptive expectations), learning refers to agents forming and refining beliefs about unknown parameters of their environment. In supervised machine learning, learning means statistical estimation, fitting the weights of a parameterized model to minimize a loss function over data. In reinforcement learning, "learning" is primarily computation. When an RL agent "learns" a Q-function, it is executing a recursive stochastic approximation algorithm to find the fixed point of the Bellman operator. We say the algorithm is "learning" because it improves its policy iteratively through simulated or real experience, but mathematically it is solving a fixed-point problem. In many applications throughout

[3] A source of confusion is *generative model*. In machine learning broadly, this means a model of the joint distribution $P(X, Y)$ or a synthetic data generator (GANs, diffusion models). In RL theory, a generative model is a simulator oracle that, given any (s, a), returns a sample $s' \sim P(\cdot|s, a)$ and reward $r(s, a)$; the usage implies random-access simulation, stronger than sequential online interaction but weaker than knowing P analytically.

[4] The closest statistical analogs are a single panel unit's time series, a realization of a stochastic process, or one "history" in a dynamic model.

this survey, particularly Section **??**, the RL algorithm is simply a numerical method for solving the Bellman equation; no actual human-like learning from experience is taking place. Finally, while RL draws inspiration from animal psychology (Section **??**), it is a drastic simplification of biological learning. Tabula rasa RL algorithms require millions of iterations of trial and error to discover policies that animals acquire rapidly. Real-world animal learning relies on innate priors, basic physical knowledge, and parental nurturing and should be distinct from RL-style "learning".

Adaptive learning in macroeconomics has used ideas formally analogous to reinforcement learning for decades, though under different names and with different motivations. In the adaptive learning literature initiated by **?**, boundedly rational agents update their beliefs about equilibrium parameters using recursive least-squares and other Robbins-Monro-type stochastic approximation algorithms. The convergence criterion in that literature, E-stability, evaluates whether the ordinary differential equation (ODE) associated with the mapping from the perceived to the actual law of motion is locally asymptotically stable at the rational expectations equilibrium. This fixed-point stability requirement is analogous to the contraction conditions governing the convergence of temporal-difference and Q-learning algorithms in RL. **?** make this mathematical connection explicit: they prove the convergence of Q-learning and actor-critic methods using the ODE method, explicitly citing **?** as a parallel application of the exact same framework to boundedly rational agents.

Active learning appears in both fields but means different things. In the inference tradition, active learning denotes Bayesian experimentation where an agent endogenously chooses what information to acquire, trading off the cost of exploration against the option value of better future decisions. In machine learning, active learning is a supervised learning protocol in which the algorithm selects which unlabeled examples to query an oracle for labels, minimizing annotation cost rather than maximizing cumulative reward. The term *exploration* is ubiquitous in RL but rarely used in the inference tradition. In RL, exploration is a modular subcomponent of a larger algorithm, a procedure for choosing informative actions that may be quite crude (such as ε-greedy random action selection) or more principled (UCB, Thompson sampling). The closest analog in the other tradition is *optimal experimentation* in the Bayesian bandit tradition (Rothschild 1974), where the value of information is derived endogenously from the agent's dynamic

program, not bolted on as a separate heuristic. The inference tradition also uses *Bayesian learning* to describe the same phenomenon, but always within a fully specified model of beliefs and preferences; in RL, exploration can be a purely algorithmic device with no decision-theoretic foundation.

Bootstrapping in statistics refers to Efron's resampling method (Efron, 1979); in RL, it means updating a value estimate using another value estimate rather than a complete realized return, as when a TD algorithm uses the target $r_{t+1}+\gamma V(s_{t+1})$ that depends on the current, uncertain estimate $V(s_{t+1})$ (Sutton, 1988). *Function approximation* in RL refers to representing value functions or policies using parameterized function classes (linear combinations of basis functions, kernel methods, or neural networks), which statisticians will recognize as sieve estimation or nonparametric series estimation, the approximation of an unknown function by projection onto a finite-dimensional basis. *Calibration* carries unrelated meanings. In the inference tradition, calibration means choosing model parameters to match a set of empirical moments or stylized facts (Kydland and Prescott 1982). In machine learning, calibration refers to probability calibration, ensuring that predicted probabilities match observed frequencies, a statistical property of a classifier's outputs.

The term *bandit* itself carries different mathematical content across disciplines. The classical multi-armed bandit in statistics (Thompson, 1933; Rothschild, 1974; Gittins, 1979) is a Bayesian sequential allocation problem. The state is the agent's posterior belief over the unknown arm distributions, and the solution is the Gittins index, an optimal allocation rule derived from the theory of optimal stopping. In the RL and computer science literature, bandits are instead framed as frequentist regret-minimization problems. Algorithms such as UCB provide worst-case bounds on cumulative regret $\sum_{t=1}^{T}(\mu^* - \mu_{A_t})$ without requiring Bayesian priors. The two traditions ask fundamentally different questions, Bayesian optimality of the full sequential problem versus minimax regret rates over adversarial or stochastic environments, and their answers are not directly comparable. Section **??** adopts the regret framework because it connects more naturally to the sample complexity concerns that arise in field experiments.

The term *contextual bandit* is especially liable to misreading. To a reader from the inference tradition, the "context" is simply the state variable x_t, and a contextual bandit looks like an MDP with an unknown reward parameter. What the RL literature signals by "context" is a specific structural restriction on transitions, the agent's action has no causal

effect on the next context, so that $P(x_{t+1} \mid x_t, a_t) = P(x_{t+1})$. This exogeneity assumption separates the exploration problem (learning which arm is best given the current context) from the planning problem (choosing actions that influence future states). When contexts evolve exogenously, there is no long-horizon credit assignment, and the problem reduces to repeated one-period optimization under uncertainty. The term "contextual" therefore flags a modeling assumption about dynamics, not merely the presence of observable covariates.

A concrete example clarifies how the two culture see a common problem. The RL literature frames some recommender systems as contextual bandits, where user covariates are the context, the recommendation is the arm, and a click or rating is the reward. One might instead view movie recommendation as a two-sided learning problem in which the platform learns user preferences while users simultaneously explore the catalog and update their own tastes. The bandit formulation absorbs the user's utility maximization into the environment's reward signal and treats user arrivals as exogenous. This is a modeling choice, not a fact about the world. Also, the object called a "bandit" in this formulation, a one-step decision under exogenous context, is a slightly different mathematical object than the Bayesian sequential allocation problem of Gittins (1979), even though both carry the same name and are related. [5] As apparent, the RL view often abstracts away from a lot of complexity in the world (human learning, human interaction) to squash the problem into the straightforward language of optimal control (π, V, P, r), while the inference tradition prefers to allow for much more complex models of human-firm interaction.

The word *policy* is itself overloaded, and the divergence runs deeper than terminology. In the inference tradition, a "policy" is a rule set by a government, central bank, or regulator, a tax schedule, a subsidy, an interest rate rule, a licensing requirement. These rules are part of the *environment* in which private agents (consumers, firms) optimize. "Policy evaluation" in this tradition means changing some aspect of the environment and asking how agents would respond: if the earned income tax credit were expanded, how would labor supply shift? The standard workflow proceeds in two steps: first estimate or calibrate the structural model (preferences, technology, market interactions), then

[5] Lattimore (2016) proves that the Gittins index with a flat Gaussian prior achieves finite-time regret of the same order as UCB, and that the index decomposes as posterior mean plus an exploration bonus that shrinks toward zero near the horizon, structurally resembling but differing from the UCB confidence bound.

simulate counterfactuals under the alternative rule. In RL, "policy" means the agent's own decision rule $\pi(a|s)$, and "policy evaluation" means computing $V^\pi(s)$, the expected return from following that rule in a fixed environment. RL typically assumes access to a high-fidelity simulator, a physics engine, a game, a digital twin, whose rules do not change; the interesting question is how the agent should behave within those rules, not what happens if the rules change. When the environment does shift, RL treats it as a nuisance (sim-to-real transfer, domain adaptation) rather than the object of study; in offline RL (Section **??**), distributional shift between the training data and the deployed environment becomes a central concern, bringing RL closer to standard counterfactual reasoning.[6]

Identification carries fundamentally different weight across the two fields. In statistics, a parameter θ is identified if it is uniquely pinned down by the combination of the data-generating process and the model's maintained assumptions; formally, no two distinct parameter values $\theta \neq \tilde{\theta}$ can generate the same distribution of observable data (**?**). Identification is a logical property of the model, not a statistical one; if identification fails, no amount of additional data or more sophisticated estimation will recover the parameter, because multiple parameter values are observationally equivalent. In RL, there is little general identification discourse. The closest analogs are narrow and domain-specific. In inverse reinforcement learning, reward identifiability asks whether the reward function can be uniquely recovered from observed optimal behavior; **?** formalize this for MaxEnt MDP models, showing that for deterministic dynamics, strong identifiability requires the domain graph to be coverable and aperiodic.[7] In model-based RL, system identification refers to learning the transition dynamics $\hat{T}$ well enough for the resulting policy to perform well, a usage inherited from control theory rather than statistics (**?**). RL researchers focus on out-of-sample performance of the learned controller, so whether parameters are identified in the statistical sense matters less; what matters is that the policy generalizes.[8]

The term *regret* itself diverges across fields. In decision theory, regret is either an emotion that modifies preferences under uncertainty (Loomes and Sugden 1982) or a

[6] A notable exception is **?**, where AlphaZero was used to evaluate alternative chess rule sets proposed by Kramnik, asking how optimal play changes under modified game rules. This is precisely an environment counterfactual.

[7] **?** explicitly note that the literature on identifying dynamic discrete choice models (**?**) addresses "an equivalent problem" to reward identifiability in IRL, providing a direct bridge between the two fields.

[8] The one RL subfield where identification is central, inverse reinforcement learning, is the subject of the companion survey (**?**).

static minimax decision criterion for choosing among actions when probabilities are unknown (Savage 1951, Manski 2004). In RL and computer science, regret is a dynamic performance metric, the cumulative gap between the agent's returns and those of the best fixed policy in hindsight, and sublinear growth of this quantity is the standard benchmark for online learning algorithms. In the other tradition, *efficiency* concerns welfare: Pareto efficiency asks whether any reallocation could improve one agent's utility without harming another, and allocative efficiency asks whether goods flow to their highest-valued uses. In RL, efficiency concerns resources: sample efficiency measures how many environment interactions an algorithm needs to find a good policy, and computational efficiency measures the operations required per timestep.

In the inference tradition, the *discount factor* is a structural parameter encoding time preference, an agent's intrinsic willingness to trade present for future consumption. **?** derived it axiomatically from preference postulates, principally stationarity and impatience, showing that these behavioral axioms uniquely imply an exponential discounted utility representation with a single discount factor strictly between zero and one (**?**). This parameter is treated as a deep primitive of preferences, to be estimated from data on intertemporal choices, and deviations from constant discounting (such as the quasi-hyperbolic present bias of Laibson 1997) are studied as substantive behavioral phenomena. In RL, the discount factor has historically served a more pragmatic role, ensuring that infinite-horizon returns remain finite and that the Bellman operator is a contraction, thereby guaranteeing convergence of dynamic programming algorithms. It is typically treated as a hyperparameter to be tuned for computational performance rather than a structural claim about the agent's preferences. **?** bridges this gap, axiomatically deriving discounting in RL from rationality postulates and showing that the fixed discount factor is best understood as an optimizing representation rather than a literal description of time preference.

0.3 Structural Equivalences

Beyond terminological differences, several formal objects in RL and the inference tradition are mathematically identical. The softmax (or Boltzmann) policy used throughout RL is the multinomial logit model of McFadden (1974). The RL softmax policy selects actions

according to

$$\pi(a \mid s) = \frac{\exp(Q(s,a)/\tau)}{\sum_{a' \in \mathcal{A}} \exp(Q(s,a')/\tau)}, \tag{1}$$

where $\tau > 0$ is a temperature parameter. In the discrete choice framework, $Q(s,a)$ plays the role of the deterministic component of utility $v(a \mid x)$, and τ is the scale parameter of the Type I extreme value (Gumbel) taste shocks ε_a. As $\tau \to 0$, the policy converges to the greedy (deterministic) policy, just as the logit choice probability concentrates on the utility-maximizing alternative as the variance of taste shocks vanishes.

The entropy regularization commonly added to RL objectives is the inclusive value (or log-sum-exp) from the discrete choice literature. The soft value function

$$V^{\text{soft}}(s) = \tau \log \sum_{a \in \mathcal{A}} \exp(Q(s,a)/\tau) \tag{2}$$

is identical to the McFadden surplus function $W(x) = \tau \log \sum_a \exp(v(a|x)/\tau) + C$, where C is Euler's constant. In the structural estimation literature, this object appears as the Emax function in dynamic discrete choice models following Rust (1987).

The advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ measures how much better action a is compared to the average action under π. In the discrete choice framework, this is precisely the choice-specific value function net of the ex-ante value function, a quantity that appears in the Hotz and Miller (1993) CCP estimator for dynamic discrete choice models.

These equivalences reflect a common convex-analytic structure. The log-sum-exp value function and the negative Shannon entropy are Fenchel conjugates of each other, so maximizing expected payoffs with an entropy bonus and recovering utilities from observed choice probabilities are two views of the same optimization. **?** build on this conjugate duality to develop identification and estimation methods for dynamic discrete choice models, and **?** show that the underlying structure extends well beyond the logit case.

The two fields arrived at this shared mathematics from opposite directions. In RL, entropy regularization began as a pragmatic trick to prevent premature convergence and encourage exploration (**?**), and only decades later did Ziebart (2010) and Levine (2018) provide decision-theoretic foundations showing that the resulting policies are robust to model misspecification. In the inference tradition, the softmax emerged not from any concern about exploration but from the random utility framework of McFadden (1974),

where agents are fully informed and choose deterministically; the apparent randomness arises entirely from the econometrician's inability to observe all relevant taste variation. The RL agent randomizes because it is ignorant of the environment; the economic agent appears to randomize because the observer is ignorant of the agent's preferences.

0.4 Notation

Table 2 maps the notation used throughout this survey to the most common econometric equivalents.

Table 2: Notation mapping between reinforcement learning and the inference tradition.

RL Term	Symbol	Inference Tradition	Symbol
State	$s \in \mathcal{S}$	State variable, covariate	x_t, Ω_t
Action	$a \in \mathcal{A}$	Choice, control variable	d_t, u_t[9]
Reward	$r(s, a)$	Per-period utility, payoff	$u(x, d)$
Discount factor	γ[10]	Discount factor	β
Policy	$\pi(a\|s)$	Decision rule, CCP	$P(d\|x)$
Value function	$V^\pi(s)$	Ex-ante value function	$\bar{V}_\theta(x)$
Q-function	$Q^\pi(s, a)$	Choice-specific value function	$v_\theta(x, d)$
Return	G_t	Present discounted value	$\sum_{k=0}^{\infty} \beta^k u_{t+k}$
Transition	$P(s'\|s, a)$	State transition law	$f(x_{t+1}\|x_t, d_t)$
Learning rate	α	Step size	α_n
TD error	δ_t[11]	Bellman residual at sample	–

References

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

John C. Gittins. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B (Methodological)*, 41(2):148–164, 1979.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Tor Lattimore. Regret analysis of the finite-horizon Gittins index strategy for multi-armed bandits. In *Proceedings of the 29th Conference on Learning Theory (COLT)*, pages 1214–1245, 2016.

Jiaxi Liu, Xiaoqing Wang, Yuming Deng, Xingyu Wu, and Yidong Zhang. Dynamic pricing on E-commerce platform with deep reinforcement learning: A field experiment. *arXiv preprint arXiv:1912.02572*, 2019.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974.

Michael Rothschild. A two-armed bandit theory of market pricing. *Journal of Economic Theory*, 9(2):185–202, 1974.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.